

Manual Técnico

GeoMIP: *Geometric Minimum Information Partition Framework*

Sistema de Análisis de Irreducibilidad Sistémica
mediante K-Particiones

Versión 2.1 (Actualización 13/06/2026)

Fecha: 2026-06-13

Equipo de Desarrollo K-QGMIP

Universidad de Caldas — Facultad de Ingeniería

Análisis y Diseño de Algoritmos — 2026-1

Autores: Manuela Vallejo Toro y Mauricio Gonzales Carmona

Índice

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN	5
1.1. Problema abordado	5
1.2. Marco teórico: Teoría de Información Integrada (IIT)	5
1.3. Justificación y relevancia	5
1.4. Alcance del documento	6
2. FUNDAMENTOS TEÓRICOS	6
2.1. Partición Mínima de Información (MIP)	6
2.2. Definición formal de k -particiones y notación de conjuntos	7
2.3. Formulación del problema de optimización combinatoria para k -particiones	8
2.4. Extensión del marco de particiones a k -particiones	9
2.4.1. Relación entre bipartición MIP y k -partición óptima	9
2.4.2. Estrategia de extensión algorítmica	9
2.5. Justificación matemática de las heurísticas Greedy y Kernighan-Lin	10
2.5.1. Heurística Greedy (extracción unilateral)	10
2.5.2. Heurística Kernighan-Lin (intercambio bilateral)	11
2.5.3. Garantía conjunta del sistema de dos fases	11
2.6. Earth Mover's Distance (EMD) y su formulación analítica	12
2.7. Espacio de búsqueda y número de Stirling	13
2.8. El hipercubo de estados y su explotación algorítmica	13
3. ARQUITECTURA DEL SOFTWARE	14
3.1. Diagrama de paquetes	14
3.2. Diagrama de clases UML	14
3.3. Diagramas de Secuencia UML	14
3.4. Patrones de diseño	14
4. ENFOQUE ALGORÍTMICO E IMPLEMENTACIÓN	15
4.1. Estrategia QNodes ($k = 2$) — Heurística Greedy	15
4.1.1. Fundamentos	15
4.1.2. Pseudocódigo	15
4.1.3. Análisis de complejidad	16
4.2. Estrategia GeometricSIA ($k = 2$) — Programación Dinámica	16
4.2.1. Fundamentos	16
4.2.2. Pseudocódigo	16
4.2.3. Análisis de complejidad	17
4.3. Estrategia KPartitionSIA — Heurística Greedy ($k = 3, 4, 5$)	17
4.3.1. Fundamentos	17
4.3.2. Pseudocódigo	18
4.3.3. Análisis de complejidad	18
4.4. Estrategia KPartitionSIA — Kernighan-Lin ($k = 3, 4, 5$)	18
4.4.1. Fundamentos	18
4.4.2. Pseudocódigo	19
4.4.3. Análisis de complejidad	19
4.5. Estrategia KPartitionSIA — Clustering Jerárquico ($k = 3, 4, 5$)	20
4.5.1. Fundamentos	20
4.5.2. Análisis de complejidad	20

4.5.3.	Limitaciones documentadas	20
4.6.	Estrategia KPartitionSIA — Espectral (baseline negativo)	20
4.6.1.	Fundamentos	20
4.6.2.	Demostración de no equivalencia con EMD	21
4.6.3.	Valor académico como cota superior	21
4.7.	Estrategia MCTS + MC-EMD ($n \leq 22$)	21
4.7.1.	Fundamentos	21
4.7.2.	Pseudocódigo	22
4.7.3.	Análisis de complejidad	22
4.8.	Estrategia KL + MC-EMD ($n = 25$)	23
4.8.1.	Fundamentos y motivación	23
4.8.2.	Pseudocódigo	24
4.8.3.	Análisis de complejidad	25
5.	MODELADO DE LA SOLUCIÓN: ARQUITECTURA REFLEJADA EN BITÁCORAS	25
5.1.	Evolución arquitectónica	25
5.2.	Sistema de caché de subsistemas	26
5.3.	Sistema de perfiles y logging	26
5.4.	Sistema de resultados (Solution)	27
5.5.	Mecanismo de timeouts adaptativos	27
6.	RESULTADOS Y CONTRIBUCIONES	27
6.1.	Resultados cuantitativos: benchmark $n = 10$	27
6.2.	Resultados cuantitativos: benchmark $n = 15$	28
6.3.	Resultados cuantitativos: benchmark $n = 20$	28
6.4.	Resultados cuantitativos: benchmark rápido $n = 25$	28
6.5.	Comparación KL vs. Greedy	29
6.6.	Comparación GeometricSIA vs. QNodes	29
6.7.	Recomendación de estrategia por tamaño de red	30
6.8.	Contribuciones e innovaciones clave	30
7.	EVALUACIÓN CRÍTICA	31
7.1.	Limitaciones técnicas y operativas	31
7.2.	Frontera computacional	31
7.3.	Análisis de escalabilidad	32
7.4.	Recomendaciones de uso y despliegue	32
7.5.	Trabajo futuro y optimizaciones propuestas	33
8.	DETALLES DE IMPLEMENTACIÓN	33
8.1.	Estructura de directorios	33
8.2.	Dependencias y plataformas	35
8.3.	Suite de pruebas	35
8.4.	Convenciones de código	36
A.	Demostración de la EMD analítica	36
B.	Tabla de complejidades completa	37

C. Referencias bibliográficas	38
D. Glosario de términos	39
E. Algoritmo KL+MC-EMD para $n \geq 25$	40

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

1.1. Problema abordado

El presente documento describe el diseño, implementación y evaluación del sistema GeoMIP (Geometric Minimum Information Partition), un marco de cómputo para el análisis de irreducibilidad sistémica en redes de transición de estados. El problema fundamental que aborda GeoMIP consiste en determinar la Partición Mínima de Información (MIP, Minimum Information Partition) de un sistema dinámico discreto, esto es, la partición de sus componentes que minimiza la pérdida de información causal bajo una métrica de distancia entre distribuciones de probabilidad.

Formalmente, dado un sistema S compuesto por n nodos con una matriz de probabilidades de transición (TPM) que define la dinámica $P(X_t | X_{t-1})$, se busca la partición π del conjunto de nodos en k subconjuntos disjuntos que minimice:

$$\phi(\pi) = \text{EMD}(P(X_t | X_{t-1}), P_\pi(X_t | X_{t-1})) \quad (1)$$

donde P_π es la distribución del sistema particionado (en el que cada parte evoluciona independientemente) y EMD es la Earth Mover's Distance entre ambas distribuciones.

Este problema pertenece a la clase de problemas de optimización combinatoria sobre espacios exponenciales: el número de k -particiones del conjunto de n elementos viene dado por el número de Stirling de segunda especie $S(n, k)$, cuyo crecimiento es superexponencial.

1.2. Marco teórico: Teoría de Información Integrada (IIT)

La Teoría de Información Integrada (IIT, por sus siglas en inglés), desarrollada por Tononi, Boly, Massimini y Koch (2016), proporciona un marco matemático para cuantificar la integración causal de un sistema. En IIT 3.0, la irreducibilidad de un sistema — su capacidad para generar información integrada, denotada como ϕ — se define como la distancia EMD entre la distribución de transición del sistema completo y la del sistema bajo su partición mínima (MIP).

El sistema GeoMIP implementa el cálculo de la MIP para k -particiones ($k = 2, 3, 4, 5$) mediante estrategias algorítmicas que van desde la búsqueda exhaustiva hasta heurísticas de optimización combinatoria, pasando por programación dinámica geométrica sobre el hipercubo de estados.

1.3. Justificación y relevancia

El cómputo exacto de la MIP tiene complejidad exponencial en el número de nodos del sistema, lo que limita su aplicación práctica a sistemas pequeños. Dado que IIT se aplica a redes neuronales, sistemas biológicos y modelos de conciencia, escalar el cálculo de la MIP a sistemas de mayor tamaño tiene implicaciones significativas tanto en neurociencia teórica como en inteligencia artificial.

GeoMIP aborda este desafío mediante:

- a) Un algoritmo de programación dinámica geométrica (GeometricSIA) que reduce la complejidad de $\Theta(N^3 \cdot 2^n)$ a $\Theta(n \cdot 2^n)$, eliminando la dependencia cúbica del número de vértices.

- b) Heurísticas de particionamiento (Kernighan-Lin, MCTS) que permiten extender el análisis a k -particiones ($k \geq 3$) con calidad de solución cercana a la óptima.
- c) Un estimador Monte Carlo de EMD(MC-EMD) que hace viable el análisis de sistemas con $n \geq 20$ nodos, donde el cómputo exacto es intratable.
- d) Un algoritmo de aproximación heurística en dos pasos (Partición Inicial Aleatoria + Refinamiento Kernighan-Lin con MC-EMD) que extiende el análisis a redes de $n = 25$ nodos, donde el método exacto excede los límites de tiempo y memoria por la explosión combinatoria.

1.4. Alcance del documento

Este manual técnico cubre:

- Los fundamentos teóricos del problema de la MIPy su relación con IIT.
- La arquitectura completa del software GeoMIP.
- El diseño algorítmico detallado de cada estrategia implementada.
- Los resultados experimentales obtenidos en los benchmarks formales, incluyendo el benchmark rápido para redes de $n = 25$.
- El algoritmo de aproximación heurística KL+MC-EMD para redes grandes.
- Las limitaciones identificadas y las recomendaciones para trabajo futuro.
- La evolución del proyecto registrada en las bitácoras de desarrollo.

2. FUNDAMENTOS TEÓRICOS

2.1. Partición Mínima de Información (MIP)

Sea S un sistema de n nodos con matriz de transición TPM definida como:

$$\text{TPM}[i, s_t, s_{t-1}] = P(X_i^t = s_t \mid X^{t-1} = s_{t-1}) \quad (2)$$

Para un estado inicial x_{t-1} y un subsistema definido por (condición, alcance, mecanismo), donde:

- **mecanismo** (M): subconjunto de nodos en $t - 1$ (presente)
- **alcance** (A): subconjunto de nodos en t (futuro)
- **condición** (C): nodos cuyo estado se fija como fondo

La distribución del sistema completo se denota como:

$$p(X_A^t \mid x_M^{t-1}, x_C^{t-1}) \quad (3)$$

La distribución del sistema particionado $\pi = \{\pi_1, \dots, \pi_k\}$ se define como el producto de las distribuciones marginales de cada parte:

$$p_\pi(X_A^t \mid x_M^{t-1}, x_C^{t-1}) = \prod_{i=1}^k p(X_{\pi_i \cap A}^t \mid x_{\pi_i \cap M}^{t-1}) \quad (4)$$

La pérdida de información causal por la partición π es:

$$\phi(\pi) = \text{EMD}(p \parallel p_\pi) \quad (5)$$

La MIPes:

$$\text{MIP} = \arg \min_{\pi \in \Pi_k} \phi(\pi) \quad (6)$$

donde Π_k es el conjunto de todas las k -particiones del subsistema.

2.2. Definición formal de k -particiones y notación de conjuntos

Sea $V = \{v_1, v_2, \dots, v_n\}$ un conjunto finito de n nodos que conforman el subsistema bajo análisis. Una k -partición de V es una familia de subconjuntos $\{P_1, P_2, \dots, P_k\}$ que satisface las siguientes propiedades:

$$(i) \text{ No vacuidad: } P_i \neq \emptyset \quad \forall i = 1, \dots, k \quad (7)$$

$$(ii) \text{ Disyunción: } P_i \cap P_j = \emptyset \quad \forall i \neq j \quad (8)$$

$$(iii) \text{ Cobertura total: } \bigcup_{i=1}^k P_i = V \quad (9)$$

El conjunto de todas las k -particiones de V se denota como $\Pi_k(V)$ y su cardinalidad está dada por el número de Stirling de segunda especie:

$$|\Pi_k(V)| = S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n \quad (10)$$

donde $\binom{k}{j} = \frac{k!}{j!(k-j)!}$ es el coeficiente binomial.

Cada parte P_i de la partición contiene dos tipos de nodos según su pertenencia temporal:

- $M_i = P_i \cap M$: subconjunto de nodos del mecanismo (presente) asignados a la parte i .
- $A_i = P_i \cap A$: subconjunto de nodos del alcance (futuro) asignados a la parte i .

La condición de partición válida requiere que cada parte tenga al menos un nodo en el mecanismo o en el alcance:

$$M_i \cup A_i \neq \emptyset \quad \forall i = 1, \dots, k \quad (11)$$

Ejemplo ilustrativo ($n = 4, k = 3$): Consideremos un sistema con nodos $V = \{A, B, C, D\}$ donde el mecanismo es $M = \{A, B\}$ y el alcance es $A = \{C, D\}$. Una 3-partición posible es:

$$\begin{aligned}
P_1 &= \{A, C\} \quad (M_1 = \{A\}, A_1 = \{C\}) \\
P_2 &= \{B\} \quad (M_2 = \{B\}, A_2 = \emptyset) \\
P_3 &= \{D\} \quad (M_3 = \emptyset, A_3 = \{D\})
\end{aligned}$$

Verificación de propiedades:

- No vacuidad: $P_1, P_2, P_3 \neq \emptyset$ [OK]
- Disyunción: $P_1 \cap P_2 = \emptyset, P_1 \cap P_3 = \emptyset, P_2 \cap P_3 = \emptyset$ [OK]
- Cobertura: $P_1 \cup P_2 \cup P_3 = \{A, B, C, D\}$ [OK]

La distribución particionada resultante para este ejemplo es:

$$\begin{aligned}
p_\pi(X_C^t, X_D^t \mid x_A^{t-1}, x_B^{t-1}) &= p(X_C^t \mid x_A^{t-1}) \times && \text{(parte } P_1) \\
& p(X_D^t) \times && \text{(parte } P_3, \text{ sin mecanismo)} \\
& 1 && \text{(parte } P_2, \text{ sin alcance)}
\end{aligned}$$

La ausencia de mecanismo en P_3 implica que la distribución marginal de X_D^t se calcula sin condicionamiento; la ausencia de alcance en P_2 implica que dicha parte no contribuye variables futuras a la distribución conjunta.

2.3. Formulación del problema de optimización combinatoria para k -particiones

El problema de encontrar la k -MIPse formula como un problema de optimización combinatoria con restricciones:

$$\begin{aligned}
&\text{Una TPMdel sistema: } T : \{0, 1\}^n \rightarrow \Delta(\{0, 1\}^n) \\
&\text{Un estado inicial: } x^{t-1} \in \{0, 1\}^n \\
\text{Dado: } &\text{Un subsistema } (C, A, M) \text{ con } C, A, M \subseteq \{1, \dots, n\} \\
&\text{Un entero } k \in \{2, 3, 4, 5\}
\end{aligned} \tag{12}$$

$$\text{Encontrar: } \pi^* = \arg \min_{\pi \in \Pi_k(V)} \phi(\pi) \tag{13}$$

$$\begin{aligned}
&(1) \pi = \{P_1, \dots, P_k\} \text{ es una } k\text{-partición de } V = A \cup M \\
&(2) P_i \neq \emptyset \quad \forall i = 1, \dots, k \\
\text{Sujeto a: } &(3) \forall i \neq j : P_i \cap P_j = \emptyset \\
&(4) \bigcup_{i=1}^k P_i = V
\end{aligned} \tag{14}$$

donde la función objetivo $\phi : \Pi_k(V) \rightarrow \mathbb{R}_{\geq 0}$ se define como:

$$\phi(\pi) = \text{EMD} \left(p(X_A^t \mid x_M^{t-1}, x_C^{t-1}) \parallel \prod_{i=1}^k p(X_{A_i}^t \mid x_{M_i}^{t-1}) \right) \tag{15}$$

con la convención de que:

- Si $A_i = \emptyset$, entonces la parte i no contribuye variables a la distribución conjunta (factor 1).
- Si $M_i = \emptyset$, entonces $p(X_{A_i}^t)$ se calcula como la distribución marginal incondicional.

Las restricciones (2)–(4) definen el espacio factible $\Pi_k(V)$. La violación de cualquiera de estas restricciones produce una partición inválida, y el costo $\phi(\pi)$ solo se define para $\pi \in \Pi_k(V)$. El algoritmo KPartitionSIA verifica estas condiciones antes de evaluar cualquier candidato; si $k > |V|$, retorna “Inviabile” inmediatamente.

2.4. Extensión del marco de particiones a k -particiones

La extensión del caso $k = 2$ (bipartición) a $k \geq 3$ (k -partición) no es trivial ni algorítmica ni matemáticamente. A continuación se presentan los fundamentos de dicha extensión.

2.4.1. Relación entre bipartición MIP y k -partición óptima

Sea π_2^* la bipartición MIP($k = 2$) del subsistema V , y sea π_k^* la k -partición óptima para $k \geq 3$. Se cumple la siguiente propiedad de monotonicidad de la pérdida:

$$\text{Para toda } \pi_k \in \Pi_k(V) \text{ obtenida refinando } \pi_2^*, \text{ se tiene: } \phi(\pi_k) \geq \phi(\pi_2^*) \quad (16)$$

Demostración: Refinar una partición consiste en subdividir alguna de sus partes en subpartes más pequeñas. Cada subdivisión adicional fuerza independencia condicional entre más subconjuntos de variables, lo que incrementa (o mantiene) la distancia EMD respecto a la distribución original. Formalmente, si π' es un refinamiento de π (es decir, cada parte de π' está contenida en alguna parte de π), entonces:

$$D_{\text{KL}}(p \parallel p_{\pi'}) \geq D_{\text{KL}}(p \parallel p_{\pi}) \quad (17)$$

y por la desigualdad de Pinsker, lo mismo aplica para la EMD.

Sin embargo, $\phi(\pi_k^*)$ puede ser **menor** que $\phi(\pi_2^*)$ cuando π_k^* no es un refinamiento de π_2^* sino una partición cualitativamente distinta que no respeta la estructura de la bipartición óptima. Esto ocurre porque:

$$\Pi_2(V) \not\subset \text{conjunto de refinamientos de ninguna } \pi_2 \text{ en particular} \quad (18)$$

Es decir, π_3^* puede explorar agrupaciones de nodos que no son extensiones de la bipartición óptima de $k = 2$.

Ejemplo concreto: en el benchmark $n = 10$, el caso ABCDEFGHIJ/ABCDEFGHIJ tiene $\phi(\pi_2^*) = 0,484$ (GeometricSIA), mientras que KL $k = 3$ encuentra $\phi = 0,000$ — una mejora del 100 % que solo es posible porque la 3-partición óptima no es un refinamiento de la bipartición óptima.

2.4.2. Estrategia de extensión algorítmica

Dado que el espacio $\Pi_k(V)$ es superexponencial, las heurísticas implementadas abordan la extensión de $k = 2$ a $k \geq 3$ mediante un enfoque constructivo iterativo:

1. Calcular la bipartición MIP π_2^* usando GeometricSIA ($k = 2$ óptimo).

2. A partir de π_2^* , construir una secuencia de particiones $\pi_2, \pi_3, \dots, \pi_k$ donde cada π_{j+1} se obtiene de π_j mediante operaciones de:
 - a *Split unilateral* (Greedy): extraer un nodo de una parte existente para formar una nueva parte.
 - b *Intercambio bilateral* (KL): intercambiar nodos entre partes existentes antes de realizar el siguiente split.
3. Evaluar la pérdida $\phi(\pi_j)$ en cada paso y retener la mejor configuración encontrada.

Esta estrategia garantiza que π_k es alcanzable desde π_2^* mediante una secuencia de movimientos elementales, pero **no garantiza** que π_k sea el óptimo global, ya que el espacio de búsqueda queda restringido a las trayectorias que parten de la bipartición óptima.

2.5. Justificación matemática de las heurísticas Greedy y Kernighan-Lin

2.5.1. Heurística Greedy (extracción unilateral)

El algoritmo Greedy implementa una estrategia de ascenso (descenso) por gradiente en el espacio de configuraciones. En cada iteración, examina todos los splits unilaterales factibles y selecciona el que minimiza el incremento en ϕ .

Fundamento teórico: Sea $f(P) = \phi(P)$ la función de pérdida para una configuración de partes $P = \{P_1, \dots, P_j\}$. La operación de split unilateral consiste en reemplazar P_i por $\{P'_i, \{v\}\}$ donde $P'_i = P_i \setminus \{v\}$. La ganancia de este split es:

$$\Delta(v, P_i) = \phi(P_{\text{actual}}) - \phi(P_{\text{nueva}}) \quad (19)$$

El algoritmo Greedy selecciona:

$$(v^*, P_i^*) = \arg \max_{v, P_i} \Delta(v, P_i) \quad (20)$$

Garantías y limitaciones:

- a *Monotonidad local:* Greedy garantiza que en cada paso la pérdida aumenta lo menos posible (no que disminuya). Esto es inherente al problema: añadir una parte siempre incrementa o mantiene la EMD.
- b *Optimalidad local:* Greedy encuentra un mínimo local en el sentido de que ningún split unilateral adicional mejora la solución, dado el orden de construcción. Sin embargo, intercambios bilaterales entre partes existentes podrían mejorarla.
- c *Cota de aproximación:* Para funciones submodulares monótonas, el algoritmo greedy tiene garantía de $(1 - 1/e)$ -aproximación. Sin embargo, $\phi(\pi)$ **no** es submodular en general, por lo que esta garantía no aplica directamente al problema de k -particiones EMD.

2.5.2. Heurística Kernighan-Lin (intercambio bilateral)

KL aborda la limitación del Greedy incorporando movimientos de intercambio que permiten escapar de los mínimos locales. La innovación clave de KL sobre Greedy es que considera el espacio de configuraciones completo mediante swaps entre pares de nodos de partes distintas.

Fundamento teórico: Dada una configuración $P = \{P_1, \dots, P_j\}$, un intercambio bilateral entre las partes P_a y P_b con nodos $u \in P_a$ y $v \in P_b$ produce una nueva configuración P' donde:

$$\begin{aligned} P'_a &= (P_a \setminus \{u\}) \cup \{v\} \\ P'_b &= (P_b \setminus \{v\}) \cup \{u\} \\ P'_i &= P_i \quad \text{para } i \neq a, b \end{aligned}$$

La ganancia del intercambio ($u \leftrightarrow v$) es:

$$\Gamma(u, v, P_a, P_b) = \phi(P) - \phi(P') \quad (21)$$

KL itera hasta que no existe ningún intercambio con $\Gamma > 0$, es decir, hasta alcanzar un equilibrio de Nash local en el espacio de configuraciones.

Garantías de calidad:

- a *Propiedad de equilibrio de Nash:* La configuración final P_{KL} tiene la propiedad de que ningún nodo puede moverse unilateralmente (ni mediante intercambio bilateral con otro nodo) para reducir ϕ . Esto es estrictamente más fuerte que la condición de Greedy (que solo garantiza optimalidad respecto a splits unilaterales).
- b *Relación de dominancia:* Sea P_G la solución de Greedy y P_{KL} la solución de KL para el mismo caso. Experimentalmente se verifica:

$$\phi(P_{KL}) \leq \phi(P_G) \quad \text{en el 100 \% de los casos probados} \quad (22)$$

Esto se debe a que P_G es alcanzable desde P_{KL} mediante movimientos unilaterales (extrayendo nodos de partes existentes), pero P_{KL} explora configuraciones adicionales que P_G no considera.

- c *Comportamiento asintótico:* A medida que el número de iteraciones de refinamiento crece, KL converge a un óptimo local en un número finito de pasos (como máximo $O(n^2)$ intercambios, ya que cada intercambio reduce estrictamente ϕ y hay un número finito de configuraciones).

2.5.3. Garantía conjunta del sistema de dos fases

El sistema GeoMIP combina ambas heurísticas en una arquitectura de dos fases:

- **Fase 1 (GeometricSIA):** Encuentra la bipartición óptima exacta ($k = 2$) con complejidad $\Theta(n \cdot 2^n)$. Para $k = 2$, GeometricSIA **no** es heurístico: explora sistemáticamente el hipercubo mediante programación dinámica y evalúa todos los candidatos relevantes.
- **Fase 2 (Greedy + KL):** Extiende a $k \geq 3$ mediante la secuencia:

- a Greedy: proporciona una solución base rápida.
- b KL: refina la solución mediante intercambios, garantizando que el resultado final es un óptimo local en el sentido fuerte (equilibrio de Nash).

Esta combinación garantiza que:

1. Para cualquier caso en $\Pi_k(V)$, el algoritmo retorna una partición que es un óptimo local en un entorno de intercambios bilaterales.
2. La calidad de la solución es monótona respecto al esfuerzo computacional: más iteraciones de KL nunca empeoran el resultado.
3. En el 100 % de los casos evaluados ($n = 10$, $n = 15$, todos los valores de k), KL produce soluciones al menos tan buenas como Greedy.

2.6. Earth Mover's Distance (EMD) y su formulación analítica

La Earth Mover's Distance entre dos distribuciones de probabilidad p y q sobre un espacio métrico (Ω, d) se define como:

$$\text{EMD}(p, q) = \min_{\gamma \in \Gamma(p, q)} \int_{\Omega \times \Omega} d(x, y) d\gamma(x, y) \quad (23)$$

donde $\Gamma(p, q)$ es el conjunto de todas las distribuciones conjuntas con marginales p y q .

Para el caso específico de la métrica Efecto (efecto EMD), cuando las variables futuras son condicionalmente independientes dado el presente, la EMD tiene una solución analítica cerrada:

$$\text{EMD}_{\text{efecto}}(p, q) = \sum_{i=1}^m |p_i - q_i| \quad (24)$$

donde m es el número de nodos en el alcance y:

$$\begin{aligned} p_i &= P(X_i^t = 0 \mid x_M^{t-1}) \\ q_i &= P(X_i^t = 0 \mid x_{\pi_i \cap M}^{t-1}) \end{aligned}$$

Esta simplificación evita la resolución del problema de transporte óptimo, reduciendo el costo de $O(2^m \log(2^m))$ a $O(m)$.

La demostración formal de esta equivalencia se presenta en el Apéndice A.

Para la métrica Causa ($\text{EMD}_{\text{causal}}$), se utiliza la distancia de Hamming como métrica de suelo:

$$\text{EMD}_{\text{causa}}(p, q) = \min_{\gamma} \sum_{s, s'} d_H(s, s') \gamma(s, s') \quad (25)$$

donde $d_H(s, s')$ es la distancia de Hamming (número de bits diferentes). Esta formulación requiere resolver el problema de transporte completo mediante el algoritmo pyemd.

2.7. Espacio de búsqueda y número de Stirling

El número de k -particiones de un conjunto de n elementos es el número de Stirling de segunda especie:

$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n \quad (26)$$

El tamaño del espacio de búsqueda para valores relevantes es:

Cuadro 1: Tamaño del espacio de búsqueda

n	$S(n, 2)$	$S(n, 3)$	$S(n, 5)$
3	3	1	—
4	7	6	1
5	15	25	10
10	511	9,330	42,525
15	16,383	$2,37 \times 10^9$	$O(10^{10})$
20	524,287	$O(10^{14})$	$O(10^{17})$

Debido a este crecimiento superexponencial, la enumeración exhaustiva es inviable incluso para $n = 10$ con $k = 5$. Las heurísticas y el enfoque de programación dinámica geométrica son indispensables.

2.8. El hipercubo de estados y su explotación algorítmica

El estado del sistema en cualquier instante es un vector binario de n bits, representable como un vértice del hipercubo $\{0, 1\}^n$. La TPM define una distribución de probabilidad sobre este hipercubo para cada estado anterior.

La estrategia GeometricSIA explota la estructura geométrica del hipercubo mediante:

- Recorrido BFS desde el estado inicial s_0 hacia el estado final $s_f = \overline{s_0}$, explorando niveles por distancia de Hamming.
- Una función de costo de transición definida como:

$$\text{tx}(i, j) = \frac{1}{2^{d_H(i, j)}} \left(|X_i - X_j| + \sum_k \text{tx}(k, j) \right) \quad (27)$$

donde X_i es el vector de variables futuras en el estado i .

- Memorización de costos de transición en tabla `transiciones`, evitando calcular trayectorias comunes.

Costo del método exacto como cuello de botella: Para redes grandes ($n \geq 20$), el método exacto `find_mip` constituye el cuello de botella principal del sistema. Su costo $O(2^{n_{\text{mec}}} \cdot n_{\text{mec}})$ se vuelve prohibitivo para mecanismos grandes: con $n_{\text{mec}} = 20$, la marginalización de NCubes requiere operaciones sobre arrays de $2^{20} = 1,048,576$ entradas.

En configuraciones con $k \geq 3$ para $n = 25$, el sistema ejecutaba `find_mip` antes de la heurística de k -partición, aun cuando dicha heurística era la vía de ejecución principal. La

corrección implementada omite completamente el paso exacto cuando $n \geq 25$, empleando directamente una partición aleatoria inicial seguida de refinamiento KL con estimación Monte Carlo de EMD(véase sección 4.8).

3. ARQUITECTURA DEL SOFTWARE

3.1. Diagrama de paquetes

El diagrama de paquetes se encuentra en la dirección: `docs/Diagramas/diagrama_paquetes.md` del repositorio.

3.2. Diagrama de clases UML

El diagrama de clases se encuentra en la dirección: `docs/Diagramas/Diagrama_De_Clases.png` del repositorio.

3.3. Diagramas de Secuencia UML

1. Escenario A – Inicialización del sistema y carga de TPM: `docs/Diagramas/DS1_Inicialización_`
2. Escenario B – Proceso iterativo de búsqueda de la k-MIP: `docs/Diagramas/DS2_Búsqueda_de_k-M`
3. Escenario C – Evaluación de una k-partición candidata: `docs/Diagramas/DS3_Evaluación_de_un`
4. Escenario D – Interacción entre componentes principales: `docs/Diagramas/DS4_Interacción_ent`

3.4. Patrones de diseño

El sistema implementa los siguientes patrones de diseño:

- a **Estrategia (Strategy Pattern)**: Las clases `GeometricSIA`, `BruteForce`, `QNodes`, `Phi`, `KPartitionSIA` implementan la interfaz común definida por la clase abstracta `SIA`, permitiendo intercambiar algoritmos sin modificar el código cliente.
- b **Singleton**: La clase `Application` en `models.base.application` implementa un Singleton que mantiene la configuración global del sistema (notación, métrica, semilla, habilitación de profiling).
- c **Fachada (Facade Pattern)**: El módulo `main.py` proporciona funciones de alto nivel (`ejecutar_desde_excel`, `ejecutar_kparticion_desde_excel`) que orquestan la interacción entre múltiples subsistemas (gestor de TPMs, estrategias, profiling, logging).
- d **Context Manager**: `ProfileContext` en `middlewares/profile.py` gestiona el ciclo de vida del profiler `PyInstrument`, garantizando inicio y finalización correctos.
- e **Decorador (Decorator Pattern)**: `profile()` y `log_execution()` son decoradores que añaden funcionalidad transversal (profiling y logging) a métodos sin modificar su código.
- f **Cache (Memoization)**: El sistema implementa dos niveles de memorización:

- a) `_SUBSYSTEM_CACHE`: evita reconstruir el subsistema para el mismo caso cuando múltiples estrategias comparten parámetros.
- b) `_FIND_MIP_CACHE`: evita recomputar la búsqueda BFS de GeometricSIA cuando KPartitionSIA la necesita como base.

4. ENFOQUE ALGORÍTMICO E IMPLEMENTACIÓN

4.1. Estrategia QNodes ($k = 2$) — Heurística Greedy

4.1.1. Fundamentos

QNodes implementa una heurística greedy de tipo submodular para la búsqueda de la bipartición óptima. El algoritmo construye iterativamente un conjunto ω (una parte de la partición) seleccionando en cada paso el nodo δ que minimiza la diferencia entre la EMD de la unión y la EMD individual de δ .

La función submodular evaluada en cada iteración es:

$$f(\omega, \delta) = \text{EMD}(\omega \cup \{\delta\}) - \text{EMD}(\{\delta\}) \quad (28)$$

que mide la “contribución marginal” de añadir δ a ω . El algoritmo selecciona el δ con la menor contribución marginal en cada paso.

4.1.2. Pseudocódigo

```

1  ALGORITHM QNodes(condicion, alcance, mecanismo, tpm):
2      subsistema <- preparar_subsistema(condicion, alcance, mecanismo
3          , tpm)
4      V <- presente[] + futuro[]           // N = m + p vertices
5      memoria_omega <- {}                 // memoizacion de EMD por
6          conjunto
7      memoria_particiones <- {}           // memoizacion de
8          particiones
9
10     PARA i <- 0 HASTA |V| - 3:
11         omega <- [V[0]]
12         deltas <- V[1:]
13         PARA j <- 0 HASTA |deltas| - 2:
14             mejor_k <- -1
15             menor_diff <- infinito
16
17             PARA k <- 0 HASTA |deltas| - 1:
18                 emd_union <- EMD(bipartir(omega UNION {deltas[k]}))
19                 emd_delta <- EMD(bipartir({deltas[k]}))
20                 diff <- emd_union - emd_delta
21                 SI diff < menor_diff:
22                     menor_diff <- diff
23                     mejor_k <- k
24
25         omega.agregar(deltas[mejor_k])
26         deltas.eliminar(deltas[mejor_k])

```



```

24
25 // Guardar candidato
26 memoria_particiones[omega] <- (emd_omega, dist_omega)
27 // Reorganizar para proxima iteracion
28 par_candidato <- omega[-1] + deltas[0]
29 omega[-1] <- par_candidato
30 V <- omega
31
32 RETORNAR argmin_{particion en memoria_particiones} perdida

```

4.1.3. Análisis de complejidad

Tiempo: $\Theta(N^3 \cdot 2^n)$

Espacio: $O(N^2 \cdot 2^n)$

donde $N = m + p$ (vértices del subsistema), $n =$ nodos totales. Tres bucles anidados sobre los vértices generan $\Theta(N^3)$ iteraciones, cada una con evaluación EMD de costo $O(2^n)$.

4.2. Estrategia GeometricSIA ($k = 2$) — Programación Dinámica

4.2.1. Fundamentos

GeometricSIA explota la geometría del hipercubo de estados para reducir la complejidad de la búsqueda. En lugar de evaluar todas las biparticiones posibles directamente, el algoritmo realiza un recorrido estructurado desde el estado inicial s_0 hasta su complemento s_f , calculando costos de transición de forma recursiva con memoización.

La función de costo recursiva se define como:

$$C(\text{nuevo}) = \sum_{i \in F} \left[y_i \cdot \left(|X_i - X_j| + \sum_{k \in \text{sucesores}} C(k) \right) \right] \quad (29)$$

donde $y_i = 1/2^{d_H(\text{prev}, \text{nuevo})}$.

El algoritmo identifica candidatos de partición examinando los caminos de costo mínimo y seleccionando aquellos puntos donde la separación de variables futuras minimiza la EMD.

4.2.2. Pseudocódigo

```

1 ALGORITHM GeometricSIA(condicion, alcance, mecanismo, tpm):
2   subsistema <- preparar_subsistema(condicion, alcance, mecanismo
3     , tpm)
4   s0 <- estado_inicial[dims_mecanismo]
5   sf <- complemento(s0)
6
7   // Inicializar tabla de transiciones (memoizacion)
8   tabla_transiciones[(s0, s0)] <- vector_cero(m)
9   caminos[0] <- [s0]

```

```

10 // Recorrido BFS por niveles de distancia Hamming
11 PARA nivel <- 1 HASTA n:
12     visitados <- {}
13     PARA cada estado_prev EN caminos[nivel-1]:
14         PARA cada bit i DONDE estado_prev[i] != sf[i]:
15             nuevo <- invertir_bit(estado_prev, i)
16             SI nuevo NOT IN visitados:
17                 costo <- calcular_costo(s0, nuevo)
18                 tabla_transiciones[(s0, nuevo)] <- costo
19                 caminos[nivel].agregar(nuevo)
20                 visitados.agregar(nuevo)
21
22 // Identificar candidatos de biparticion
23 candidatos <- []
24 costos_totales <- tabla_transiciones[(s0, sf)]
25 PARA idx <- 0 HASTA m-1:
26     candidatos.agregar( (todos_presentes, todos_futuros \ {idx
27                           }) )
28 PARA nivel <- 1 HASTA n//2:
29     candidatos.agregar( mejor_estado_nivel(caminos[nivel]) )
30
31 // Evaluar candidatos
32 PARA cada (presentes, futuros) EN candidatos:
33     dist <- bipartir(futuros, presentes).distribucion_marginal
34     ()
35     perdida <- emd_efecto(dist, dist_marginal_subsistema)
36     memoria_particiones[(presentes, futuros)] <- perdida
37
38 RETORNAR argmin_{(p,f) en memoria_particiones} perdida

```

4.2.3. Análisis de complejidad

Tiempo: $\Theta(n \cdot 2^n)$

Espacio: $O(n \cdot 2^n)$

La reducción respecto a QNodes ($\Theta(N^3 \cdot 2^n)$) es significativa: se elimina el factor cúbico N^3 , dejando solo el factor exponencial 2^n inherente al espacio de estados.

Speedup empírico: $2,8\times$ ($n = 10$) a $4,1\times$ ($n = 15$) sobre QNodes.

4.3. Estrategia KPartitionSIA — Heurística Greedy ($k = 3, 4, 5$)

4.3.1. Fundamentos

Extiende la bipartición de GeometricSIA a k particiones mediante un proceso iterativo de extracción unilateral de nodos. Parte de la MIP($k = 2$) encontrada por GeometricSIA y divide recursivamente las partes existentes extrayendo nodos para formar nuevas partes.

En cada iteración, el algoritmo:

1. Toma cada parte P de las partes actuales.

2. Para cada nodo v en P (tanto presente como futuro), crea una parte nueva $P_2 = \{v\}$ y $P_1 = P \setminus \{v\}$.
3. Evalúa la pérdida EMD_{total} de la configuración resultante.
4. Aplica el split que minimiza la pérdida.
5. Repite hasta alcanzar k partes.

4.3.2. Pseudocódigo

```

1  ALGORITHM KPartitionSIA_Greedy(condicion, alcance, mecanismo, tpm,
   mip):
2      GeometricSIA.find_mip()                //  $O(n \cdot 2^n)$ 
3      mip <- memoria_particiones.min()
4      partes <- [parte1(mip), parte2(mip)]
5
6      MIENTRAS |partes| < k:
7          mejor_split <- null
8          mejor_perdida <- infinito
9
10         PARA cada parte P EN partes:
11             PARA cada nodo v EN P.presentes UNION P.futuros:
12                 P1 <- P \ {v}
13                 P2 <- {v}
14                 partes_test <- partes con reemplazo(P, [P1, P2])
15                 perdida <- SUMA_i EMD(bipartir(partes_test[i]))
16                 SI perdida < mejor_perdida:
17                     mejor_perdida <- perdida
18                     mejor_split <- (P, P1, P2)
19
20             aplicar_split(mejor_split)
21
22     RETORNAR partes con menor perdida EMD total

```

4.3.3. Análisis de complejidad

Tiempo: $O(k^3 \cdot n \cdot 2^n)$

Espacio: $O(2^n)$

La dependencia k^3 proviene de los bucles anidados sobre partes actuales y nodos, repetidos para cada nivel hasta alcanzar k partes.

4.4. Estrategia KPartitionSIA — Kernighan-Lin ($k = 3, 4, 5$)

4.4.1. Fundamentos

Kernighan-Lin (KL) mejora la heurística greedy al incorporar intercambios bilaterales entre partes. En lugar de solo extraer nodos (movimiento unilateral), KL considera

intercambiar pares de nodos entre partes existentes, lo que permite escapar de mínimos locales.

El algoritmo:

1. Parte de la bipartición MIPde GeometricSIA.
2. Bucle de refinamiento: para cada par de partes (P_i, P_j) , prueba intercambiar cada par de nodos $(u \in P_i, v \in P_j)$ y calcula la ganancia en EMD.
3. Aplica el intercambio con mayor ganancia positiva.
4. Repite hasta que ningún intercambio mejora la solución.
5. Si se requieren más partes, divide la parte más grande y repite.

4.4.2. Pseudocódigo

```

1  ALGORITHM KPartitionSIA_KL(condicion, alcance, mecanismo, tpm, k):
2      GeometricSIA.find_mip()
3      mip <- memoria_particiones.min()
4      partes <- [parte1(mip), parte2(mip)]
5
6      MIENTRAS |partes| < k:
7          // Refinamiento KL: intercambios bilaterales
8          MEJORA <- True
9          MIENTRAS mejora:
10             mejora <- False
11             PARA cada par (P_i, P_j) EN partes x partes, i < j:
12                 PARA cada (u en P_i, v en P_j):
13                     perdida_actual <- evaluar_partes(partes)
14                     partes_prueba <- intercambiar(P_i, P_j, u, v)
15                     perdida_nueva <- evaluar_partes(partes_prueba)
16                     ganancia <- perdida_actual - perdida_nueva
17                     SI ganancia > 0:
18                         aplicar_intercambio(partes, u, v)
19                         mejora <- True
20
21             // Split de la parte mas grande para nueva iteracion
22             IF |partes| < k:
23                 P_grande <- max(partes, key=size)
24                 mejor_split <- mejor_split_unilateral(P_grande)
25                 aplicar_split(mejor_split)
26
27      RETORNAR partes con menor perdida EMD total

```

4.4.3. Análisis de complejidad

Tiempo: $O(k \cdot n^2 \cdot 2^n \cdot \text{iter}_{\text{KL}})$

Espacio: $O(2^n)$

donde iter_{KL} es el número de iteraciones de refinamiento (típicamente 5–20). Aunque KL es más costoso que Greedy en cada iteración, la calidad superior de la solución compensa el costo adicional.

Relación empírica calidad/tiempo:

- $n = 10$: KL $1.4\times$ más lento, 79 % mejor pérdida ($k = 3$)
- $n = 15$: KL $1.1\times$ más lento, 66 % mejor pérdida ($k = 3$)

4.5. Estrategia KPartitionSIA — Clustering Jerárquico ($k = 3, 4, 5$)

4.5.1. Fundamentos

Construye una matriz de costos de transición a partir de la tabla generada por GeometricSIA y aplica clustering jerárquico de Ward para agrupar los nodos en k clusters. La matriz aumentada $A = [[0, C], [C^T, 0]]$ captura las relaciones entre nodos presentes y futuros.

4.5.2. Análisis de complejidad

Tiempo: $O(n \cdot 2^n + n^3)$

Espacio: $O(n^2)$

El clustering (pdist + linkage Ward) tiene costo $O(n^3)$, dominado por el término $O(n \cdot 2^n)$ de la fase de GeometricSIA.

4.5.3. Limitaciones documentadas

El clustering jerárquico minimiza la distancia euclidiana entre perfiles de nodos, no la EMD. Esto produce pérdidas 9–10× peores que Greedy:

$$n = 10, k = 3 : \quad \text{Greedy} = 0,165, \quad \text{Clustering} = 1,536$$

Esta heurística se mantiene exclusivamente como comparación negativa y demostración de que la similitud geométrica no equivale a la minimización de EMD.

4.6. Estrategia KPartitionSIA — Espectral (baseline negativo)

4.6.1. Fundamentos

Implementa el algoritmo de clustering espectral normalizado (Ng, Jordan & Weiss, 2001) sobre la matriz de afinidad derivada de los costos EMD individuales por nodo. Construye el Laplaciano normalizado:

$$L_{\text{sym}} = D^{-1/2}(D - W)D^{-1/2} \quad (30)$$

donde $W[i, j] = \exp(-|\text{costo}_i - \text{costo}_j|/\sigma)$, obtiene los k primeros vectores propios y aplica k -means en el espacio espectral.

4.6.2. Demostración de no equivalencia con EMD

El particionamiento espectral resuelve el problema de relajación continua del mínimo corte normalizado (Normalized Cut, Shi & Malik, 2000):

$$\min_{A \subset V} \frac{\text{cut}(A, \bar{A})}{\text{vol}(A)} + \frac{\text{cut}(A, \bar{A})}{\text{vol}(\bar{A})} \quad (31)$$

El problema que requiere el proyecto es:

$$\min_{\pi \in \Pi_k} \text{EMD}(p_\pi \parallel p_{\text{sistema}}) \quad (32)$$

Demostración por contraejemplo: en el caso ABCDEFGHIJ/ABCDEFGHIJ ($n = 10$, $k = 3$), la partición de mínimo corte espectral produce $\text{EMD} = 4,992$, mientras que KL encuentra una partición con $\text{EMD} = 0,000$. La EMD de una k -partición no es aditiva sobre aristas del grafo, por lo que el Laplaciano no puede capturarla.

4.6.3. Valor académico como cota superior

El spectral se mantiene como baseline negativo documentado que:

- a Demuestra empíricamente que algoritmos correctos para similitud estructural no son transferibles directamente a distancia probabilística.
- b Proporciona una cota superior útil: cualquier heurística que supere al spectral en pérdida EMD ya es una contribución válida.
- c Ilustra el trade-off tiempo/calidad: spectral $O(n^3)$ es rápido pero produce pérdidas inaceptables; KL es más lento pero produce pérdidas óptimas.

4.7. Estrategia MCTS + MC-EMD ($n \leq 22$)

4.7.1. Fundamentos

Para sistemas con $n \geq 20$, donde el costo exponencial de la evaluación exacta de EMD es prohibitivo, se implementa una estrategia de dos capas:

1. **Capa 1 — MC-EMD (Estimación Monte Carlo de EMD):** Reemplaza la evaluación exacta de EMD sobre los $2^{n_{\text{mec}}}$ estados del subsistema por un promedio sobre S muestras aleatorias uniformes:

$$\text{EMD}_{\text{MC}}(p, q) = \frac{1}{S} \sum_{s=1}^S |p(s) - q(s)| \quad (33)$$

Error: $O(1/\sqrt{S})$ — insesgado, controlable. Speedup: $2^{n_{\text{mec}}}/S$ — para $n_{\text{mec}} = 14$, $S = 2000$: $\sim 8\times$.

2. **Capa 2 — MCTS (Monte Carlo Tree Search):** Reemplaza la búsqueda local de KL por una búsqueda global basada en árbol. Cada nodo del árbol representa una asignación parcial de nodos a k partes. La exploración se guía por Upper Confidence Bound (UCB):

$$\text{UCB}(v) = Q(v) + c \sqrt{\frac{\ln(N_{\text{parent}})}{N_v}} \quad (34)$$

donde $Q(v)$ es la recompensa media, N_v el número de visitas y $c = \sqrt{2}$ la constante de exploración.

4.7.2. Pseudocódigo

```

1  ALGORITHM MCTS_MCEMD(condicion, alcance, mecanismo, tpm, k,
2                      n_iter=300, n_samples=2000):
3      subsistema <- preparar_subsistema(...)
4      raiz <- Nodo(asignacion_vacia, visitas=0, recompensa=0)
5
6      PARA iter <- 1 HASTA n_iter:
7          nodo <- raiz
8
9          // Fase 1: Selecccion
10         MIENTRAS nodo NO expandido Y nodo NO terminal:
11             nodo <- UCB_seleccion(nodo.hijos, c=1.41)
12
13         // Fase 2: Expansion
14         SI nodo NO terminal:
15             nodo <- expandir(nodo)
16
17         // Fase 3: Simulacion (rollout)
18         asignacion <- copia(nodo.asignacion)
19         MIENTRAS asignacion INCOMPLETA:
20             nodo_restante <- nodos_sin_asignar(asignacion)
21             v <- seleccion_aleatoria(nodo_restante)
22             parte <- seleccion_aleatoria({1..k})
23             asignacion[v] <- parte
24
25         // Fase 4: Evaluacion con MC-EMD
26         perdida <- MC_EMD(asignacion, subsistema, n_samples)
27
28         // Fase 5: Retropropagacion
29         MIENTRAS nodo != null:
30             nodo.visitas += 1
31             nodo.recompensa += (-perdida)
32             nodo <- nodo.padre
33
34         asignacion_optima <- mejor_asignacion(raiz)
35     RETORNAR perdida_optima, particion_optima

```

4.7.3. Análisis de complejidad

Tiempo: $O(I \cdot n \cdot k \cdot S)$

Espacio: $O(n \cdot k)$

donde I = iteraciones MCTS, S = número de muestras MC-EMD. Para $n = 22$, $k = 3$, $I = 300$, $S = 2000$: $\sim 39,6$ M operaciones numpy, frente a $\sim 64,8$ M operaciones Python de KL exacto (con ventaja adicional de vectorización numpy $\sim 1000\times$ por operación).

4.8. Estrategia KL + MC-EMD ($n = 25$)

4.8.1. Fundamentos y motivación

Para redes de $n = 25$, el enfoque algorítmico exacto original excede los límites de tiempo y memoria debido a la explosión combinatoria. La TPMde 25 nodos contiene $2^{25} = 33,554,432$ filas ($\sim 3,4$ GB en float32), y el método exacto `find_mip` (GeometricSIA) requiere un recorrido BFS sobre el hipercubo de estados con marginalización de NCubes, cuyo costo escala como $O(2^{n_{mec}})$ por evaluación. Con mecanismos de 20–25 nodos, cada evaluación demanda tiempo prohibitivo.

Cuello de botella identificado: Para configuraciones con $k \geq 3$, el sistema ejecutaba el método exacto lento (`find_mip`) antes de activar la heurística, incluso cuando la heurística era la vía de ejecución principal. Esto causaba retrasos masivos (varios minutos) solo en la fase de inicialización por cada caso de prueba. La solución fue omitir completamente el paso exacto para $n = 25$, ejecutando directamente una heurística rápida estructurada en dos fases:

1. **Fase 1 — Partición Inicial Aleatoria:** División inicial del mecanismo en k grupos al azar. Los nodos presentes y futuros del subsistema se mezclan mediante un shuffle controlado (`numpy.random.Generator.shuffle`) y se distribuyen equitativamente en k partes, asegurando que cada parte tenga al menos un nodo.
2. **Fase 2 — Refinamiento Kernighan-Lin con MC-EMD:** Optimización progresiva mediante el intercambio de nodos entre grupos. En cada iteración, se evalúa la reducción de pérdida al mover un nodo de su parte actual a otra parte. El refinamiento continúa hasta que ningún movimiento mejora la pérdida (equilibrio de Nash local) o se alcanza el máximo de iteraciones (8–20 según el tamaño del mecanismo).

Muestreo Monte Carlo (MC-EMD): Para evitar el costo $O(2^{n_{mec}})$ de la evaluación exacta de EMD en cada movimiento, se emplea un estimador Monte Carlo completamente vectorizado que evalúa la EMD sobre S estados muestreados uniformemente del hipercubo. El error del estimador es $O(1/\sqrt{S})$ — insesgado y controlable. Para $n = 25$ se usan $S = 2500$ muestras por evaluación, ofreciendo un speedup de $\sim 2^{n_{mec}}/S \sim 350\times$ respecto al cálculo exacto para $n_{mec} = 20$.

Implementación en el código: La heurística se implementa en la función `_heuristica_kpart_kl` de `kpartition.py` (`src/controllers/strategies/kpartition.py`). El método recibe el número de partes deseado, realiza el split aleatorio y aplica `_refinar_kl()` con evaluación interna basada en `_loss_interna()`, que utiliza MC-EMD cuando n_{mec} supera el umbral configurado. La pérdida final reportada puede ser exacta o aproximada según el parámetro `perdida_mc_final`.

El flujo de decisión en `_resolver_k_partition()` bifurca la estrategia según el tamaño de red:

- Para $n \geq 25$ y `forzar_heuristica="kl_mc"`, se invoca directamente `_solution_kl_mc_rapida()` omitiendo por completo `find_mip` y la tabla de costos de GeometricSIA.
- Para $n \leq 22$ en el benchmark rápido se emplea MCTS+MC-EMD (sección ??), que también omite `find_mip` pero utiliza búsqueda en árbol en lugar de refinamiento local.

El benchmark rápido (`benchmark_rapido.py`) aplica esta lógica: para $n \geq 25$ configura KPartitionSIA con `forzar_heuristica="kl_mc"`, `n_samples_mc=2500`, y `perdida_mc_final=True`; para $n \leq 22$ emplea `forzar_heuristica="mcts"` con parámetros agresivos por tamaño de red.

Parámetros por defecto según tamaño de mecanismo (n_{mec}):

Cuadro 2: Parámetros MC-EMD por tamaño de mecanismo

n_{mec}	Muestras S	máx $_{iter}$
≤ 12	Evaluación exacta	—
≤ 16	800	12
≥ 18	2000–2500	8

4.8.2. Pseudocódigo

```

1  ALGORITHM KL_MC_Rapida(k, n_samples=2500):
2      nodos_presentes <- dims_ncubos
3      nodos_futuros <- indices_ncubos
4      todos_nodos <- [('p',v) for v en presentes]
5                  + [('f',v) for v en futuros]
6
7      SI |todos_nodos| < k:
8          RETORNAR particion_invalida
9
10     // Fase 1: Particion inicial aleatoria
11     rng.shuffle(todos_nodos)
12     base <- floor(|todos_nodos| / k)
13     extra <- |todos_nodos| MOD k
14     tamanos <- [base + 1 si i < extra else base
15                 PARA i=0 HASTA k-1]
16
17     partes <- particionar(todos_nodos segun tamanos)
18
19     // Fase 2: Refinamiento Kernighan-Lin con MC-EMD
20     PARA iter <- 1 HASTA max_iter:
21         mejora <- False
22         perdida_actual <- MC_EMD(partes, n_samples)
23
24         PARA cada par (P_i, P_j), i != j:
25             PARA cada nodo u EN P_i:
26                 partes_prueba <- mover(u, P_i -> P_j)
27                 nueva_perdida <- MC_EMD(partes_prueba,
28                                         n_samples)
29                 SI nueva_perdida < perdida_actual - epsilon:
30                     partes <- partes_prueba
31                     mejora <- True
32                     BREAK
33
34     SI NOT mejora:
35         BREAK

```

```

36
37     perdida_final <- SI perdida_mc_final:
38         MC_EMD(partes, n_samples)
39         SINO:
40             _evaluar_particion(partes)
41     RETORNAR Solution(particion=partes,
42                       perdida=perdida_final)

```

4.8.3. Análisis de complejidad

Tiempo: $O(\text{máx_iter} \cdot k \cdot n \cdot S)$

Espacio: $O(n + S)$

donde:

- máx_iter = iteraciones de refinamiento (8–20)
- k = número de partes (2–5)
- n = nodos del subsistema
- S = muestras MC-EMD (800–2500)

La complejidad es polinomial en todos los parámetros, eliminando el factor exponencial $2^{n_{\text{mec}}}$ de las estrategias exactas. El speedup teórico frente a GeometricSIA es $O(2^{n_{\text{mec}}} / (k \cdot S \cdot \text{máx_iter}))$.

Speedup empírico ($n = 25$, $k = 2$):

- GeometricSIA exacto: inviable (timeout > 24 h)
- KL + MC-EMD: ~ 47 s por caso

5. MODELADO DE LA SOLUCIÓN: ARQUITECTURA REFLEJADA EN BITÁCORAS

5.1. Evolución arquitectónica

La arquitectura final del sistema es el resultado de una evolución registrada en 36 bitácoras de desarrollo (16–18 mayo y 12–13 junio 2026), cuyos hitos principales son:

- **Fase 0 (Diagnóstico, Bitácora 01):** Verificación de que `geometric.py` reproduce la Tabla 4.2 del documento GeoMIP original (N3C, error $< 10^{-4}$). Corrección de bug en `BruteForce` (falta de argumento `tpm` en `sia_preparar_subsistema`). Corrección de `np.infty` obsoleto a `np.inf` en `force.py`.
- **Fase 1 (Refactorización, Bitácoras 02–04):** Archivado de archivos experimentales (`hyper-v*.py`) a `_archive/`. Estandarización de la clase `Solution` con campos `n_nodos`, `k`, `tiempo_ms`. Suite de 34 tests de regresión con 100 % de paso. Corrección de bug en `q_nodes.py`.

- **Fase 2 (Validación, Bitácoras 05–06):** Validador de 27 TPMs (N3–N22) con formato correcto. Documentación de límites prácticos en README.
- **Fase 3 (Benchmark, Bitácoras 07–13):** Creación de `benchmark.py` con medición de tiempo, memoria, timeout. Corrección de timeout para Windows (threading en lugar de SIGALRM). Benchmark $n = 10$ (49 casos, 3 min, 100 % convergencia). Integración de BruteForce para validación cruzada ($n \leq 6$).
- **Fase 4 (Heurísticas y Optimizaciones, Bitácoras 14–27):** Análisis de complejidad asintótica. Implementación de heurística Kernighan-Lin. Implementación de baseline espectral. Caché de subsistemas. Implementación MCTS + MC-EMD. Benchmarks $n = 10$, $n = 15$, $n = 20$.
- **Fase 5 (Escalabilidad a $n = 25$, Bitácoras 28–36):** Prueba de QNodes $k = 2$ en $n = 25$ con carga streaming float32. Diseño del algoritmo de aproximación KL+MC-EMD. Implementación y ejecución de benchmark aproximado. Benchmark rápido MCTS+MC-EMD para $n \leq 22$. Ejecución completa $n = 25$ con KL+MC-EMD: 50 casos, ~ 67 min.

5.2. Sistema de caché de subsistemas

Implementado en SIA (`src/models/base/sia.py`) para evitar la reconstrucción redundante del mismo subsistema cuando múltiples estrategias procesan el mismo caso.

- Clave de caché: (`id(tpm)`, `condicion`, `alcance`, `mecanismo`)
- Mecanismo: diccionario de módulo `_SUBSYSTEM_CACHE` con copia profunda para seguridad.
- Limpieza: `limpiar_cache_subsistemas()` entre casos de benchmark.

Impacto: reduce 8 construcciones de subsistema por caso (7 estrategias + 1) a 1 construcción + 7 deep-copies (~ 1 ms cada una).

5.3. Sistema de perfiles y logging

El logging (`middlewares/slogger.py`) proporciona:

- Salida coloreada por consola via `colorama`.
- Rotación de logs por fecha en directorios estructurados.
- Decorador `log_execution()` para trazabilidad automática.

El profiling (`middlewares/profile.py`) proporciona:

- Profiling por función via `PyInstrument`.
- Context manager `ProfilerContext` para control granular.
- Decorador `profile()` para profiling selectivo.
- Salida HTML con línea de tiempo.

Para benchmark con $n \geq 20$, el logging automático genera archivos separados `stdout/stderr` con marca temporal en `logs/n20_plus/`.

5.4. Sistema de resultados (Solution)

La clase `Solution` (`src/models/core/solution.py`) encapsula:

- **estrategia**: nombre del algoritmo utilizado
- **perdida** (ϕ): valor EMDmínimo encontrado
- **particion**: representación textual de la partición
- **tiempo_ejecucion**: tiempo de CPU en segundos
- **n_nodos**: número de nodos del sistema
- **k**: número de particiones
- **distribuciones**: distribuciones original y particionada

Propiedades computadas: `tiempo_ms = tiempo_ejecucion * 1000`.

La salida a Excel consolida todas las estrategias por fila de caso, facilitando el análisis comparativo.

5.5. Mecanismo de timeouts adaptativos

El benchmark implementa timeouts configurables por estrategia y tamaño de red:

Cuadro 3: Timeouts por tamaño de red

Tamaño	Timeout por estrategia
$n \leq 10$	120 s
$n \leq 15$	300 s
$n \leq 20$	1800 s (mejorado a 21600 s en versión final)
$n > 20$	86400 s (máximo configurable)

El mecanismo utiliza `threading.Thread + join(timeout=value)`, funcional en Windows (a diferencia de `signal.SIGALRM`).

Cada estrategia que excede el timeout se marca como “X” (TIMEOUT) en los resultados, permitiendo distinguir entre convergencia exitosa (“.”) y fallo por límite de tiempo.

6. RESULTADOS Y CONTRIBUCIONES

6.1. Resultados cuantitativos: benchmark $n = 10$

- Fuente: `benchmark_completo_2026-05-17_16h56.xlsx`
- Casos: 49
- Red: N10A (1024 filas)
- Estado: 100 % convergencia en todas las estrategias
- Resultados en: `GeoMIP/data/results/n10`

6.2. Resultados cuantitativos: benchmark $n = 15$

- Fuente: `benchmark_completo_2026-05-17_16h56.xlsx`
- Casos: 50
- Red: N15A (32768 filas)
- Estado: 100 % convergencia en todas las estrategias
- Resultados en: `GeoMIP/data/results/n15`

Las pérdidas en $n = 15$ son $\sim 60\times$ menores que en $n = 10$ porque las distribuciones de probabilidad sobre $2^{15} = 32768$ estados son naturalmente más difusas, lo que reduce la diferencia absoluta entre distribuciones original y particionada.

6.3. Resultados cuantitativos: benchmark $n = 20$

- Fuente: `benchmark_completo_2026-05-18_04h38.xlsx`
- Casos: 50
- Red: N20A (1048576 filas)
- Timeout: 21600 s por estrategia
- Estado: 100 % de casos completados (convergencia o timeout documentado)
- Resultados en: `GeoMIP/data/results/n20`

6.4. Resultados cuantitativos: benchmark rápido $n = 25$

- Fuente: `n25_rapido_2026-06-13_09h37.xlsx`
- Casos: 50
- Red: N25A (33,554,432 filas, $\sim 3,4$ GB)
- Algoritmo: KL + MC-EMD (aproximado, `forzar_heuristica="kl_mc"`)
- Parámetros: $S = 2500$ muestras MC-EMD, `perdida_mc_final=True`, `máx_iter = 8`
- Estado: 50/50 ok en $k = 2, 3, 4, 5$ (200 mediciones)
- Tiempo total de corrida: ~ 67 min (incluye una sola carga de TPM)
- Resultados en: `GeoMIP/data/results/n25`

Notas de interpretación:

- Las pérdidas son aproximadas (MC-EMD), no exactas como en `benchmark.py`.
- $k = 2$ tarda más porque el mecanismo completo (25 nodos) pesa en KL+MC.

- $k = 4$ y $k = 5$ son más rápidos en promedio (menos iteraciones KL efectivas o mecanismos más chicos).
- A mayor k , la pérdida media sube: trade-off esperado donde particiones más finas incrementan la independencia condicional forzada.
- No existe benchmark exacto de referencia para $n = 25$ (inviabile).

6.5. Comparación KL vs. Greedy

Cuadro 4: Proporción de casos donde KL supera a Greedy

k	$n = 10$ (49 casos)			$n = 15$ (50 casos)		
	KL mejor	Empate	Greedy	KL mejor	Empate	Greedy
3	47/49	2	0	5/19	14	0
4	48/49	1	0	7/19	12	0
5	49/49	0	0	12/18	6	0

Conclusión: KL **nunca** produce una solución peor que Greedy.
Mejora porcentual promedio de KL sobre Greedy:

- $n = 10$, $k = 3$: 79.2 % mejor pérdida (KL $1.4\times$ más lento)
- $n = 10$, $k = 4$: 59.6 % mejor pérdida (KL $1.8\times$ más lento)
- $n = 10$, $k = 5$: 52.2 % mejor pérdida (KL $2.3\times$ más lento)
- $n = 15$, $k = 3$: 65.9 % mejor pérdida (KL $1.1\times$ más lento)
- $n = 15$, $k = 4$: 31.2 % mejor pérdida (KL $1.2\times$ más lento)
- $n = 15$, $k = 5$: 42.5 % mejor pérdida (KL $1.5\times$ más lento)

6.6. Comparación GeometricSIA vs. QNodes

Para $k = 2$, GeometricSIA produce pérdida prácticamente idéntica a QNodes con una fracción del tiempo de cómputo:

- $n = 10$: Geo=0.076590 vs QN=0.079699 (Geo 4 % mejor, $2.8\times$ más rápido)
- $n = 15$: Geo=0.001263 vs QN=0.001263 (idénticos, Geo $4.1\times$ más rápido)

Conclusión: Para $k = 2$, siempre preferir GeometricSIA sobre QNodes.

6.7. Recomendación de estrategia por tamaño de red

Cuadro 5: Estrategia recomendada según tamaño de red

Tamaño	Estrategia recomendada	
$n \leq 10$	KPartitionSIA-KL $k = 3$	82 % casos óptimos c
$11 \leq n \leq 15$	GeometricSIA $k = 2$ primero, luego KL $k = 3$ si necesario	62 % casos: $k = 2$ su
$16 \leq n \leq 20$	MCTS + MC-EMD	Evaluación exacta invi
$n = 22$	MCTS + MC-EMD (benchmark rápido)	50 c
$n = 25$	KL + MC-EMD (aproximado)	Único viable; partición alea

Para $k = 2$ específicamente: GeometricSIA siempre sobre QNodes. Para $k \geq 3$: KL siempre sobre Greedy cuando sea computacionalmente viable. Para $n = 25$: la evaluación es inherentemente aproximada; no existe referencia exacta para validación cruzada.

6.8. Contribuciones e innovaciones clave

1. **Algoritmo GeometricSIA**: Programación dinámica geométrica sobre el hipercubo de estados que reduce la complejidad de búsqueda de MIP de $\Theta(N^3 \cdot 2^n)$ a $\Theta(n \cdot 2^n)$, con speedup empírico de $2.8\text{--}4.1\times$ sobre QNodes.
2. **Heurística Kernighan-Lin para k -particiones en contexto IIT**: Aplicación original del algoritmo de intercambio bilateral al problema de MIP, demostrando superioridad consistente sobre la heurística greedy.
3. **MCTS + MC-EMD**: Combinación de Monte Carlo Tree Search con estimación Monte Carlo de EMD que extiende la viabilidad computacional de $n \leq 15$ a $n = 20$, con error cuantificable $O(1/\sqrt{S})$.
4. **KL + MC-EMD para $n = 25$** : Algoritmo de aproximación heurística en dos pasos (partición aleatoria + refinamiento KL con MC-EMD) que permite analizar redes de 25 nodos donde el método exacto es inviable, completando 50 casos en ~ 67 minutos.
5. **Demostración académica**: Prueba empírica de que el clustering espectral (normalized cut) es inadecuado para minimizar EMD en sistemas IIT, estableciendo una cota superior negativa.
6. **Sistema de caché multinivel**: Memoización de subsistemas y resultados de `find_mip` que elimina redundancia computacional entre estrategias, reduciendo hasta $8\times$ el costo de preparación por caso.
7. **Carga streaming de TPMs** (`tpm_io.py`): Función `load_tpm_csv` que lee TPMs línea a línea en float32, evitando MemoryError de `genfromtxt` en $n \geq 25$ (archivos de $\sim 3,4$ GB).

7. EVALUACIÓN CRÍTICA

7.1. Limitaciones técnicas y operativas

- a **Escalabilidad exponencial:** Todos los algoritmos implementados tienen complejidad que incluye el factor 2^n (espacio de estados del subsistema). Para $n > 20$, el costo computacional es prohibitivo en hardware de CPU estándar.
- b **Dependencia de la métrica Efecto:** La EMDanalítica (EMD_{efecto}) *solo es validaba jo el supuesto de independencia*
- b **Heurísticas no garantizan optimalidad:** Ni Greedy ni KL garantizan encontrar la partición óptima global. KL supera sistemáticamente a Greedy pero puede quedar atrapado en mínimos locales.
- c **MCTS + MC-EMD produce resultados aproximados:** El error de estimación $O(1/\sqrt{S})$ es controlable pero no nulo. Para $S = 2000$, el error típico es $\sim 2\%$.
- d **Limitación de k :** El sistema soporta $k = 2, \dots, 5$. Particiones con $k > 5$ incrementan la pérdida EMD sin beneficio compensatorio en los conjuntos de datos evaluados.
- e **Compatibilidad con PyPhi:** La integración con `pyphi` requiere parches de compatibilidad para Python 3.10+ (`collections.abc`).

7.2. Frontera computacional

Cuadro 6: Frontera computacional observada experimentalmente

n	TPM filas	Tiempo/caso obs.	Estrategia viable
10	1,024	~ 10 s	Todas (QN, Geo, KL, G)
15	32,768	~ 35 s	Todas (QN, Geo, KL, G)
20	1,048,576	$\sim 7,3$ min media	Geo + MCTS + MC-EMD
22	4,194,304	$> 3,5$ h (to)	Solo MC-EMD (aproximado)
25	33,554,432	~ 67 min total (50 casos)	KL + MC-EMD (aproximado)

(to) = timeout

Frontera práctica confirmada:

- $n \leq 15$: completamente viable (segundos a minutos)
- $n = 20$: viable con optimizaciones (horas)
- $n = 22$: viable solo con aproximación MCTS+MC-EMD
- $n = 25$: viable solo con aproximación KL+MC-EMD (resultados aproximados)

7.3. Análisis de escalabilidad

El cuello de botella computacional en todos los algoritmos es la operación de marginalización de NCubes (`bipartir()` → `marginalizar()` → `np.mean()`).

Para $n = 22$ con subsistema de 14 nodos sobre TPM de 4M filas:

- 21 biparticiones candidatas × 14 NCubes por partición × 4M entradas por NCube
~ $1,2 \times 10^9$ operaciones de punto flotante por nivel BFS.
- Con hasta 14 niveles BFS → ~ $1,7 \times 10^{10}$ operaciones por caso.
- A ~ 10^8 operaciones/s en numpy (CPU single-core): ~ 170 s por nivel → inviable sin paralelismo u optimización adicional.

El crecimiento observado es coherente con el modelo teórico:

- $n = 10 \rightarrow n = 15$: $3.2\times$ medido ($32\times$ esperado por 2^n) — mejor de lo esperado porque los subsistemas no escalan al máximo en todos los casos.
- $n = 15 \rightarrow n = 20$: ~ $60\times$ medido ($32\times$ esperado) — coherente con subsistemas más grandes y mayor overhead de gestión de memoria.

7.4. Recomendaciones de uso y despliegue

a Selección de algoritmo:

- Para $n \leq 10$: usar KPartitionSIA con heurística KL y $k = 3$.
- Para $11 \leq n \leq 15$: usar GeometricSIA $k = 2$; si se requiere $k > 2$, usar KL con timeout adecuado.
- Para $n = 20$: usar benchmark con timeout ≥ 21600 s, priorizando GeometricSIA y MCTS+MC-EMD para $k > 2$.
- Para $n = 22$: usar benchmark rápido (MCTS+MC-EMD) con timeout ≥ 480 s por caso.
- Para $n = 25$: usar benchmark rápido con KL+MC-EMD (`forzar_heuristica="kl_mc"`, `muestras=2500`, `perdida_mc_final=True`). Los resultados son inherentemente aproximados.

b Configuración de hardware:

- RAM mínima: 16 GB para $n \leq 15$, 32 GB para $n = 20$.
- CPU: se beneficia de múltiples núcleos, aunque la implementación actual es single-core para el núcleo algorítmico.
- Disco: SSD recomendado para carga de TPMs grandes (N20A: ~ 1M filas).

c Ejecución:

- Usar `uv sync` para entorno reproducible.
- Para benchmarks: `uv run python benchmark.py -n 10 15 20`.
- Para $n \geq 20$: incluir `-timeout 21600` para permitir convergencia.
- Monitorear logs en `results/logs/` durante ejecuciones largas.

d Validación:

- Ejecutar suite de tests: `uv run pytest tests/`.
- Verificar convergencia cruzada BruteForce en $n \leq 6$.
- Para resultados críticos, ejecutar múltiples semillas.

7.5. Trabajo futuro y optimizaciones propuestas

Las siguientes optimizaciones (Fase 4 del plan de proyecto) están identificadas pero no implementadas:

Cuadro 7: Optimizaciones propuestas

Optimización	Impacto estimado	Complejidad
Representación sparse (<code>scipy.sparse</code> en NCubes)	5–20× speedup en marginalizar	Alta
Paralelización de biparticiones candidatas	$\sim 7\times$ speedup por caso	Media
float32 en NCube.data	2× speedup en ops numpy	Baja
Limitación de purview a ≤ 10 nodos por caso	Permite $n > 20$ costo de exact.	Baja
GPU (CUDA) para marginalización vectorizada	10–50× speedup	Alta

Se estima que la combinación de todas las optimizaciones permitiría alcanzar $n = 22$ de forma viable con el algoritmo exacto. $n = 25$ ya es viable con la aproximación KL+MC-EMD, aunque el algoritmo exacto (GeometricSIA) requeriría un enfoque distribuido (cluster o GPU) debido al tamaño de la TPM (33M filas).

8. DETALLES DE IMPLEMENTACIÓN

8.1. Estructura de directorios

```
GeoMIP/
src/
  tpm_io.py                # Carga streaming float32 de TPMs
  run_qnodes_k2.py         # QNodes k=2 para n=25 (legacy)
  geomip_paths.py         # Rutas absolutas del proyecto
  benchmark_rapido.py      # Benchmark rapido MCTS/KL+MC-EMD
  benchmark_aprox.py       # Benchmark aproximado KL+MC-EMD
  gen_comparativa.py       # Generacion de tablas comparativas
Method2_Dynamic_Programming_Reformulation/
  exec.py                 # Punto de entrada
  run_kpart.py            # Ejecucion batch k-particion
  debug_kpart.py          # Depuracion KPartitionSIA
  pyproject.toml          # Dependencias
  pyphi_config.yml        # Config PyPhi
src/
  main.py                 # Orquestador principal
  constants/
    base.py               # Constantes globales
```

error.py	# Mensajes de error
models.py	# Constantes de modelos
controllers/	
manager.py	# Gestor de TPMs
strategies/	
force.py	# BruteForce (exhaustiva)
geometric.py	# GeometricSIA (DP)
kpartition.py	# KPartitionSIA (k>=3)
phi.py	# PyPhi (IIT)
q_nodes.py	# QNodes (greedy)
funcs/	
base.py	# EMD, Hamming, binario
format.py	# Formateo de particiones
system.py	# Particiones/combos
middlewares/	
profile.py	# Profiling PyInstrument
slogger.py	# Logging coloreado
models/	
base/	
application.py	# Singleton configuracion
sia.py	# SIA abstracto + cache
core/	
ncube.py	# NCube (hipercubo)
solution.py	# Resultados
system.py	# Sistema/TPM
enums/	
distance.py	# Metricas
notation.py	# Notaciones
data/	
samples/	# TPMs (N3-N22)
validate_tpms.py	# Validador
data/results/	
n10/, n15/, n20/, n22/, n25/	# Resultados por red
aprox/	# Resultados benchmark aprox. KL+MC
rapido/	# Resultados benchmark rapido
comparativa/	# Tablas comparativas
finales/	# Resultados consolidados historicos
logs/	
n10_n15/	# Logs de benchmarks n<=15
n20_plus/	# Logs de benchmarks n>=20
historico/	# Versiones anteriores
_archive/	# Pruebas muy antiguas
README.txt	# Documentacion de la estructura
tests/	
confptest.py	# Config pytest

test_emd.py	# Tests EMD
test_geometric_n3.py	# Tests N3
test_kpart_coherence.py	# Tests k-particion
test_kpartition_eval.py	# Tests evaluacion k-particion (+14)
test_solutions.py	# Tests Solution
docs/	
ManualTecnico_KQMIP.tex	# Manual LaTeX (historico)
gen_tables.py	# Generador tablas LaTeX
tables/	# Tablas LaTeX generadas
bitacoras/	# 36 bitacoras de desarrollo

8.2. Dependencias y plataformas

Requerimientos mínimos:

- Python $\geq 3,9,13$
- numpy (manipulación de arrays)
- pandas, openpyxl (Excel)
- pyphi (integración IIT)
- pyemd (cómputo EMDcompleto)
- pyinstrument (profiling)
- colorama (terminal coloreada)
- pyttsx3 (anuncios por voz)

Para redes grandes ($n \geq 21$), la función `load_tpm_csv` (`tpm_io.py`) reemplaza a `numpy.genfromtxt`, leyendo el archivo CSV línea a línea en float32 con un buffer de 8 MB. Esto evita MemoryError al cargar TPMs de 25 nodos ($\sim 33,5$ M filas, $\sim 3,4$ GB). La función `load_tpm` selecciona automáticamente el método según el tamaño de la red (`stream_threshold=21`).

Plataformas soportadas:

- Windows 10/11 (validado)
- Linux (compatibilidad verificada)
- macOS (no probado, pero debería funcionar con dependencias estándar)

8.3. Suite de pruebas

La suite de tests (pytest) consta de 48 pruebas organizadas en 5 módulos:

- `test_emd.py` (7 tests): Verificación de `emd_efecto` analítica (casos triviales y bordes).

- `test_geometric_n3.py` (10 tests): Reproducción de la Tabla 4.2 del documento GeoMIP (N3C). Validación cruzada GeometricSIA vs BruteForce (N3A). Verificación de consistencia de resultados.
- `test_kpart_coherence.py` (6 tests): Verificación de k correcto y n_nodos en resultados. Validación de particiones inviables ($k > \text{nodos totales}$).
- `test_solutions.py` (11 tests): Campos mínimos presentes en todas las estrategias. Tipos correctos de cada campo. Consistencia de la propiedad `tiempo_ms`.

Ejecución: `uv run pytest tests/`

8.4. Convenciones de código

- Nomenclatura: `snake_case` para funciones y variables, `PascalCase` para clases, `SCREAMING_CASE` para constantes.
- Tipado: anotaciones de tipo Python (mypy compatible).
- Logging: uso de `SafeLogger` con formato estructurado.
- Documentación: comentarios en español (consistente con el equipo).
- Gestión de errores: excepciones con mensajes descriptivos en `src/constants/error.py`.
- Archivos de datos: CSV para TPMs, Excel para configuraciones y resultados, YAML para configuración de librerías externas.

A. Demostración de la EMD analítica

Para la métrica Efecto, la EMD entre dos distribuciones de probabilidad sobre variables binarias condicionalmente independientes tiene solución cerrada.

Dado un sistema con m variables futuras condicionalmente independientes dado el presente:

$$P(X_1, \dots, X_m \mid x^{t-1}) = \prod_{i=1}^m P(X_i \mid x^{t-1}) \quad (35)$$

Sean $p_i = P(X_i = 0 \mid x^{t-1})$ y $q_i = P(X_i = 0 \mid x_\pi^{t-1})$ las probabilidades marginales de OFF para cada variable bajo la distribución original y particionada respectivamente.

La EMD con distancia L_1 como métrica de suelo entre dos distribuciones producto sobre $\{0, 1\}^m$ es:

$$\text{EMD}(p, q) = \sum_{i=1}^m |p_i - q_i| \quad (36)$$

Demostración:

La distribución conjunta sobre $\{0, 1\}^m$ es:

$$p(s) = \prod_{i=1}^m ((1 - p_i)^{1-s_i} \cdot p_i^{s_i}) \quad (37)$$

La distancia L_1 en el espacio de estados es $d(s, t) = \sum_i |s_i - t_i|$.

El acoplamiento óptimo (optimal coupling) que minimiza el costo de transporte para distribuciones producto es el acoplamiento independiente:

$$\gamma(s, t) = p(s) \cdot q(t) \quad (38)$$

Para este acoplamiento:

$$\begin{aligned} \sum_{s, t} d(s, t) \gamma(s, t) &= \sum_i \sum_{s_i, t_i} |s_i - t_i| p_i(s_i) q_i(t_i) \\ &= \sum_i [p_i(0)q_i(1) + p_i(1)q_i(0)] \\ &= \sum_i [p_i(1 - q_i) + (1 - p_i)q_i] \\ &= \sum_i |p_i - q_i| \end{aligned} \quad (39)$$

Por la desigualdad de Kantorovich-Rubinstein, este acoplamiento es óptimo para distancias L_1 en espacios producto, lo que completa la demostración.

B. Tabla de complejidades completa

Cuadro 8: Complejidades de todos los algoritmos implementados

Algoritmo Clase	Tiempo	Espacio
QNodes $k = 2$ Exponencial	$\Theta(N^3 \cdot 2^n)$	$O(N^2 \cdot 2^n)$
GeometricSIA $k = 2$ Exponencial*	$\Theta(n \cdot 2^n)$	$O(n \cdot 2^n)$
BruteForce $k = 2$ Factorial	$O(S(n, 2) \cdot 2^n)$	$O(2^n)$
KPart-Greedy $k \geq 3$ Exponencial*	$O(k^3 \cdot n \cdot 2^n)$	$O(2^n)$
KPart-KL $k \geq 3$ Exponencial*	$O(k \cdot n^2 \cdot 2^n \cdot \text{it}_{\text{KL}})$	$O(2^n)$
KPart-Clustering $k \geq 3$ Exponencial*	$O(n \cdot 2^n + n^3)$	$O(n^2)$
KPart-Spectral $k \geq 3$ Exponencial*	$O(n \cdot 2^n + n^3)$	$O(n^2)$
MCTS + MC-EMD $k \geq 2$ Polinomial en S	$O(I \cdot n \cdot k \cdot S)$	$O(n \cdot k)$
KL + MC-EMD $k \geq 2$ ($n \geq 25$) Polinomial en S	$O(\text{it}_{\text{KL}} \cdot k \cdot n \cdot S)$	$O(n + S)$

* El factor 2^n es inherente al hipercubo de estados; no existe algoritmo polinomial exacto para MIP(problema NP-hard en general).

Donde:

- n = nodos del subsistema (purview + mecanismo)
- N = vértices totales (presentes + futuros)
- k = número de particiones
- I = iteraciones MCTS
- S = muestras MC-EMD
- it_{KL} = iteraciones de refinamiento KL

Nota: KL+MC-EMD y MCTS+MC-EMD son las únicas estrategias con complejidad polinomial en todos sus parámetros, al eliminar el factor $2^{n_{\text{mec}}}$ mediante muestreo Monte Carlo. Esto las hace aptas para $n \geq 22$ donde las estrategias exactas son inviables.

C. Referencias bibliográficas

1. Tononi, G., Boly, M., Massimini, M., Koch, C. (2016). Integrated information theory: from consciousness to its physical substrate. *Nature Reviews Neuroscience*, 17(7), 450–461.
2. Shi, J., Malik, J. (2000). Normalized Cuts and Image Segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(8), 888–905.
3. Ng, A., Jordan, M., Weiss, Y. (2001). On Spectral Clustering: Analysis and an algorithm. *NeurIPS 2001*.
4. Kernighan, B.W., Lin, S. (1970). An Efficient Heuristic Procedure for Partitioning Graphs. *Bell System Technical Journal*, 49(2), 291–307.
5. Kocsis, L., Szepesvári, C. (2006). Bandit Based Monte-Carlo Planning. *ECML 2006*, 282–293.
6. Mayner, W.G.P., Marshall, W., Albantakis, L., Findlay, G., Marchman, R., Tononi, G. (2018). PyPhi: A toolbox for integrated information theory. *PLOS Computational Biology*, 14(7), e1006343.
7. Rubner, Y., Tomasi, C., Guibas, L.J. (2000). The Earth Mover’s Distance as a Metric for Image Retrieval. *International Journal of Computer Vision*, 40(2), 99–121.
8. Cover, T.M., Thomas, J.A. (2006). *Elements of Information Theory* (2nd ed.). Wiley-Interscience.
9. Pearl, J. (2009). *Causality: Models, Reasoning, and Inference* (2nd ed.). Cambridge University Press.

D. Glosario de términos

EMD(Earth Mover’s Distance): Distancia entre distribuciones de probabilidad que mide el costo mínimo de transformar una distribución en otra.

IIT(Integrated Information Theory): Teoría que cuantifica la integración causal de un sistema mediante la información integrada ϕ .

MIP(Minimum Information Partition): Partición de un sistema que minimiza la pérdida de información causal, utilizada para calcular ϕ .

TPM(Transition Probability Matrix): Matriz de probabilidades de transición del sistema: define $P(X_t | X_{t-1})$ para cada posible estado.

NCube: Estructura de datos que representa una distribución de probabilidad sobre el hipercubo $\{0, 1\}^n$ con operaciones de condicionamiento y marginalización.

SIA (System Irreducibility Analysis): Clase abstracta que define la interfaz común para todas las estrategias de búsqueda de particiones.

MC-EMD: Estimación Monte Carlo de la Earth Mover’s Distance, que evalúa la EMD sobre un subconjunto aleatorio de estados en lugar del espacio completo.

MCTS(Monte Carlo Tree Search): Algoritmo de búsqueda heurística para espacios de decisión grandes, que combina exploración (UCB) con simulación aleatoria.

KL (Kernighan-Lin): Algoritmo de particionamiento de grafos que utiliza intercambios bilaterales de nodos entre particiones para minimizar una función de costo.

UCB (Upper Confidence Bound): Estrategia de selección en MCTS que balancea exploración y explotación mediante la fórmula $Q(v) + c\sqrt{\ln(N_{\text{parent}})/N_v}$.

K-Partición: Familia de k subconjuntos disjuntos y no vacíos cuya unión cubre el conjunto total de nodos del sistema.

Equilibrio de Nash (en particionamiento): Estado en el que ningún nodo puede moverse unilateralmente (ni mediante intercambio bilateral con otro nodo) para reducir la pérdida EMD de la partición actual.

Partición Inicial Aleatoria: Estrategia de inicialización que asigna nodos a k partes mediante un shuffle controlado, utilizada como punto de partida para el refinamiento KL en redes grandes ($n = 25$).

Benchmark rápido: Ejecución del sistema con parámetros agresivos (MCTS+MC-EMD o KL+MC-EMD) que omite el paso exacto `find_mip` para obtener resultados aproximados en tiempo reducido.

Carga streaming (float32): Técnica de lectura de TPMs línea a línea en precisión float32, implementada en `tpm_io.py`, que evita MemoryError al cargar archivos de gran tamaño ($n \geq 25$, $\sim 3,4$ GB).

E. Algoritmo KL+MC-EMD para $n \geq 25$

El algoritmo KL+MC-EMD para $n \geq 25$ (sección 4.8) se estructura en dos fases:

1. **Fase 1: Partición Inicial Aleatoria.** Se mezclan los nodos del subsistema mediante `numpy.random.Generator.shuffle` y se distribuyen en k partes de tamaño equitativo, asegurando que cada parte tenga al menos un nodo. Esta fase tiene costo $O(n)$.
2. **Fase 2: Refinamiento Kernighan-Lin con estimación Monte Carlo.** Se iteran intercambios de nodos entre partes, evaluando la pérdida mediante MC-EMD con $S = 2500$ muestras. El refinamiento converge cuando ningún movimiento mejora la pérdida o se alcanza `máx_iter` = 8 iteraciones. Esta fase tiene costo $O(\text{máx_iter} \cdot k \cdot n \cdot S)$.

La omisión del paso exacto `find_mip` (GeometricSIA) reduce la complejidad de $O(2^n)$ a $O(\text{poli}(n, k, S))$, haciendo viable el análisis de redes de 25 nodos. La pérdida reportada es aproximada (MC-EMD), con error controlable $O(1/\sqrt{S})$.